

遠回りナビゲーションシステム コード規約

Version 1.0

作成日: 2025年6月17日

作成者: 01班

対象言語: TypeScript（フロントエンド）, JavaScript（バックエンド）, React, Node.js

目次

- 規約の目的
- ファイルヘッダー規則
- 命名規則
- TypeScript規則（フロントエンド）
- JavaScript規則（バックエンド）
- エラーハンドリング
- Git運用規則
- 品質チェック

1. 規約の目的

1.1 なぜコード規約が必要なのか

- 可読性向上: 誰が書いたコードでも理解しやすい
- 保守性向上: バグ修正や機能追加が効率的
- チーム協力: 個人のクセを排除し、統一された開発スタイル

1.2 適用範囲

- フロントエンド（React + TypeScript）
- バックエンド（Node.js + Express + JavaScript）
- テストコード

2. ファイルヘッダー規則

2.1 必須ヘッダー情報

すべてのソースファイルに以下のヘッダーを記載

javascript

```
/**
 * @fileoverview ファイルの概要を1行で記述
 * @description 詳細な説明
 * @author 作成者名
 * @created YYYY-MM-DD
 * @updated YYYY-MM-DD
 * @version X.Y.Z
 */
```

2.2 TypeScript例

typescript

```
/**
 * @fileoverview 地図表示コンポーネント
 * @description Leafletを使用した地図表示機能
 * @author 佐藤花子
 * @created 2025-06-15
 * @updated 2025-06-17
 * @version 1.2.0
 */
```

2.3 JavaScript例

javascript

```
/**
 * @fileoverview 経路計算API
 * @description OSRM APIを使用した経路計算機能
 * @author 田中一郎
 * @created 2025-06-16
 * @updated 2025-06-17
 * @version 1.1.0
 */
```

2.4 バージョン更新ルール

- メジャー更新 (1.0.0 → 2.0.0) : API変更など互換性のない変更
- マイナー更新 (1.0.0 → 1.1.0) : 新機能追加
- パッチ更新 (1.0.0 → 1.0.1) : バグ修正

3. 命名規則

3.1 基本パターン

対象	形式	例
変数・関数	camelCase	<code>userName</code> , <code>calculateRoute()</code>
クラス・型・コンポーネント	PascalCase	<code>UserService</code> , <code>RouteData</code> , <code>MapDisplay</code>
定数	SCREAMING_SNAKE_CASE	<code>MAX_DISTANCE_KM</code>
ファイル	camelCase	<code>userService.js</code> , <code>routeTypes.ts</code>

3.2 具体例

```
javascript

//  良い例
const userName = "田中太郎";
const MAX_RETRY_COUNT = 3;

function calculateDistance(point1, point2) { }

class RouteService { }

//  悪い例
const user_name = "田中太郎"; // snake_case使わない
const maxretrycount = 3;      // 小文字のみ使わない
function route() { }           // 動詞がない
```

4. TypeScript規則（フロントエンド）

4.1 型の明示

typescript

//  型を明確に定義

```
interface RouteRequest {  
  readonly origin: GeographicPoint;  
  readonly destination: GeographicPoint;  
  readonly maxDuration: number;  
}  
  
function searchRoute(request: RouteRequest): Promise<Route> {  
  // 実装  
}
```

//  any型は避ける

```
function searchRoute(request: any): any {  
  // 避けるべき  
}
```

4.2 React コンポーネント

typescript

```
interface MapProps {  
  readonly center: GeographicPoint;  
  readonly zoom: number;  
  readonly onLocationSelect?: (point: GeographicPoint) => void;  
}  
  
const MapDisplay: React.FC<MapProps> = ({ center, zoom, onLocationSelect }) => {  
  const handleClick = (event: React.MouseEvent) => {  
    // 処理  
  };  
  
  return <div onClick={handleClick}>地図</div>;  
};
```

5. JavaScript規則（バックエンド）

5.1 JSDocによる型情報

javascript

```
/**
 * 二点間の距離を計算します
 * @param {Object} point1 - 起点の座標
 * @param {number} point1.latitude - 緯度
 * @param {number} point1.longitude - 経度
 * @param {Object} point2 - 終点の座標
 * @returns {number} 距離 (メートル単位)
 * @throws {Error} 座標が無効な場合
 */
function calculateDistance(point1, point2) {
  if (!isValidPoint(point1) || !isValidPoint(point2)) {
    throw new Error('Invalid coordinates');
  }
  // 実装
}
```

5.2 Express API

javascript

```
/**
 * 経路検索API
 * @param {Object} req - リクエストオブジェクト
 * @param {Object} res - レスポンスオブジェクト
 */
async function searchRoutes(req, res) {
  try {
    const { origin, destination } = req.body;

    // バリデーション
    if (!origin || !destination) {
      return res.status(400).json({ error: 'Origin and destination required' });
    }

    // 経路計算
    const routes = await calculateRoutes(origin, destination);
    res.json({ routes });

  } catch (error) {
    console.error('Route search error:', error);
    res.status(500).json({ error: 'Internal server error' });
  }
}
```

6. エラーハンドリング

6.1 基本パターン

javascript

```
//  適切なエラーハンドリング
async function saveData(data) {
  try {
    // バリデーション
    if (!data) {
      throw new Error('Data is required');
    }

    // 処理実行
    const result = await database.save(data);
    return result;

  } catch (error) {
    // ログ出力
    console.error('Save error:', error.message);

    // 再スロー（必要に応じて）
    throw new Error('Failed to save data');
  }
}
```

6.2 カスタムエラー

javascript

```
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = 'ValidationError';
  }
}

// 使用例
if (!email.includes('@')) {
  throw new ValidationError('Invalid email format');
}
```

7. Git運用規則

7.1 コミットメッセージ

型(スコープ): 件名

例:

feat(auth): ユーザーログイン機能を追加

fix(map): 地図表示のバグを修正

docs(readme): インストール手順を更新

型の種類:

- feat: 新機能
- fix: バグ修正
- docs: ドキュメント更新
- style: コードスタイル修正
- refactor: リファクタリング
- test: テスト追加

7.2 ブランチ命名

feature/機能名	# 新機能開発
bugfix/修正内容	# バグ修正
hotfix/緊急修正	# 緊急対応

8. 品質チェック

8.1 コードレビューチェック項目

必須項目:

- ☐ ファイルヘッダー（作成者・日付・バージョン）記載済み
- ☐ 命名規則準拠
- ☐ エラーハンドリング実装済み
- ☐ 適切なコメント記載

TypeScript固有:

- ☐ 型定義が適切
- ☐ any型を避けている

JavaScript固有:

- ☐ JSDocによる型情報記載
- ☐ 入力値検証実装済み

8.2 自動チェック設定

package.json

```
json
{
  "scripts": {
    "lint": "eslint src/",
    "lint:fix": "eslint src/ --fix",
    "format": "prettier --write src/"
  }
}
```

8.3 規約違反時の対応

1. 軽微な違反: 警告とコメント
2. 重要な違反: 修正要求
3. 重大な違反: マージ拒否

運用のポイント

チーム内での活用方法

1. 新メンバー加入時: 必ず本規約を読んでもらう
2. 開発開始前: チーム全体で規約の内容を確認
3. コードレビュー時: 規約に基づいてチェック
4. 定期見直し: 月1回程度、規約の改善点を議論

段階的な導入

1. 第1週: ファイルヘッダーと命名規則
2. 第2週: エラーハンドリングとコメント
3. 第3週: Git運用規則
4. 第4週: 品質チェックの自動化

本規約は学習と実践を通じて、継続的に改善していく生きた文書です。